Preparar el entorno y ejecutar pruebas con pytest sin Jupyter

Índice

1. Situarse en la carpeta del proyecto . . . 1
2. (si no lo has creado ya)Crear el entorno virtual . . . 2
3. Instalar pytest dentro del entorno . . . 2
4. Comprobar la instalación sin activar el entorno . . . 2
5. Ejecutar las pruebas desde la carpeta del proyecto . . . 3
    5.1. Ejecutar un solo archivo . . . 3
    5.2. Ejecutar varios archivos . . . 3
    5.3. Ejecutar una sola función de prueba . . . 4
    5.4. Ejecutar pruebas cuyo nombre contenga una palabra . . . 4
    5.5. Ejecutar todos los ejemplos correctos del notebook . . . 4
    5.6. Ejecutar todos los tests de la carpeta . . . 4
    5.7. Detenerse en el primer fallo . . . 4
    5.8. Mostrar un resumen más breve . . . 5
6. Interpretar los resultados . . . 5
7. Terminar la sesión . . . 5
8. Volver a trabajar otro día . . . 5

Estas instrucciones son para Linux/macOS y suponen que `uv` ya está instalado.

IMPORTANTE	***Instalaremos pytest únicamente en el entorno virtual `venv_edc_py`, no en el Python del sistema. Ejecutaremos los comandos con `uv run`, sin activar ni desactivar el entorno.***

1. Situarse en la carpeta del proyecto

Abre una terminal en la carpeta que contiene **`testing_chap07/`**.

Todos los comandos de este documento se ejecutan desde esa misma carpeta del proyecto, sin entrar en **`testing_chap07/`**.

La estructura, después de crear el entorno virtual, será similar a esta:

```
mi_proyecto/
├── chap07_pytest.ipynb
├── .venv_edc_py/
```

```
└── testing_chap07/
    ├── funciones.py
    ├── test_basicos.py
    ├── test_parametrizados.py
    └── ...
```

> **NOTA** *Si trabajas con el notebook, ejecuta primero las celdas `%%writefile` correspondientes: son las que crean los archivos de funciones y pruebas. Después de modificar una de esas celdas, vuelve a ejecutarla para guardar los cambios.*

2. (si no lo has creado ya)Crear el entorno virtual

Desde la carpeta del proyecto:

```
uv venv .venv_edc_py
```

Este paso solo se hace la primera vez. No necesitas volver a crear el entorno cada vez que trabajes.

3. Instalar pytest dentro del entorno

Indicamos explícitamente el Python del entorno para evitar instalar en otro lugar:

```
uv pip install --python .venv_edc_py/bin/python pytest
```

> **IMPORTANTE** *No uses la opción `--system`: queremos mantener la instalación dentro del entorno virtual.*

> **NOTA** *`monkeypatch` y `capsys` forman parte de pytest. No necesitas instalarlos por separado. `unittest.mock` forma parte de la biblioteca estándar de Python.*

4. Comprobar la instalación sin activar el entorno

```
uv run --no-project --python .venv_edc_py/bin/python python -m pytest --version
```

En los comandos de ejecución:

- `uv run` ejecuta un comando.
- `--no-project` evita que uv seleccione o sincronice un proyecto detectado en esta

carpeta o en sus carpetas superiores.
- `--python .venv_edc_py/bin/python` indica el intérprete del entorno virtual que queremos utilizar.
- `python -m pytest` ejecuta pytest con ese intérprete.

Puedes comprobar qué intérprete se está usando:

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -c "import sys; print(sys.executable)"
```

La ruta mostrada debe terminar en `.venv_edc_py/bin/python`.

CONSEJO	*No necesitas ejecutar `source .venv_edc_py/bin/activate`. Cada comando selecciona explícitamente el entorno que debe utilizar.*

5. Ejecutar las pruebas desde la carpeta del proyecto

En todos los ejemplos mantenemos la terminal en la carpeta que contiene tanto `.venv_edc_py/` como `testing_chap07/`.

Las rutas de los archivos de pruebas empiezan por `testing_chap07/`.

NOTA	*La barra invertida `\` al final de una línea permite continuar el comando en la siguiente. No escribas espacios después de ella. También puedes escribir todo el comando en una sola línea, eliminando esas barras.*

5.1. Ejecutar un solo archivo

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -v testing_chap07/test_basicos.py
```

La opción `-v` muestra el nombre y el resultado de cada caso de prueba.

5.2. Ejecutar varios archivos

Escribe sus rutas separadas por espacios:

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -v \
  testing_chap07/test_basicos.py \
  testing_chap07/test_parametrizados.py
```

5.3. Ejecutar una sola función de prueba

Utiliza `::` para separar el archivo y el nombre de la función:

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -v \
  testing_chap07/test_basicos.py::test_encuentra_una_letra
```

5.4. Ejecutar pruebas cuyo nombre contenga una palabra

Por ejemplo, seleccionar las pruebas relacionadas con una edad:

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -v testing_chap07/ -k "edad"
```

5.5. Ejecutar todos los ejemplos correctos del notebook

Esta selección excluye el fallo intencional y los ejercicios sin resolver:

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -v \
  testing_chap07/test_basicos.py \
  testing_chap07/test_parametrizados.py \
  testing_chap07/test_excepciones.py \
  testing_chap07/test_input.py \
  testing_chap07/test_print.py \
  testing_chap07/test_consola.py \
  testing_chap07/test_mock_print.py
```

5.6. Ejecutar todos los tests de la carpeta

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -v testing_chap07/
```

Pytest buscará los archivos de pruebas en **`testing_chap07/`** y sus subcarpetas.

ADVERTENCIA	*En este material también encontrará* **`test_error_intencional.py`** *y los tests de los ejercicios que hayas guardado. Es normal que aparezcan fallos mientras no hayas corregido el ejemplo intencional o completado esos ejercicios.*

5.7. Detenerse en el primer fallo

```
uv run --no-project --python .venv_edc_py/bin/python \
```

```
  python -m pytest -v -x testing_chap07/
```

5.8. Mostrar un resumen más breve

Sustituye `-v` por `-q`:

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -q testing_chap07/test_basicos.py
```

6. Interpretar los resultados

- **PASSED**: la prueba ha pasado.
- **FAILED**: una comprobación no se ha cumplido o se ha producido un fallo durante la prueba.
- **ERROR**: ha ocurrido un problema, por ejemplo al importar un archivo o preparar una prueba.

Si una prueba falla, lee el informe y compara el resultado obtenido con el esperado. No cambies la expectativa únicamente para ocultar un error del programa.

7. Terminar la sesión

Puedes cerrar la terminal directamente. No necesitas ejecutar `deactivate`, porque no hemos activado el entorno.

El entorno virtual y sus paquetes permanecen guardados en `.venv_edc_py/`.

8. Volver a trabajar otro día

Abre una terminal en la carpeta del proyecto y ejecuta:

```
uv run --no-project --python .venv_edc_py/bin/python \
  python -m pytest -v testing_chap07/test_basicos.py
```

No necesitas volver a crear el entorno, reinstalar pytest ni activar el entorno virtual.